The National Higher School of Artificial Intelligence

Introduction to AI Course Project

Spring 2026

Algiers in 24 Hours:

A Time-Constrained Orienteering Approach to Tourist Route Optimisation

Full Name Group

Abstract

Provide a brief overview of the project to introduce the reader to your report.

1. Introduction

A brief introduction to the project problem you are addressing, contextualising the problem, motivating it, and explaining its importance.

2. State of the Art

3. Dataset Building

3.1 Data Collection

We created a dataset of 58 tourist landmarks in Algiers, classified into five landmark categories: historical, attractions, traditional and artistic, shopping, and religious sites.

Each entry in the dataset has GPS coordinates, the time the site is open to visitors, an estimate for how long it takes to visit, and a user rating from either Google Maps or TripAdvisor. Public sources were used to check the hours of operation, although much of the data was obtained from Google Maps directly. Table 1 presents the attributes, attribute types, and methods used to obtain the data.

Table 1: Dataset attribute descriptions.

Attribute	Type	Source	Description
Name	Text	Manually	Full official name of the landmark
Category	Text	Manually	Thematic group assigned to one of five classes
Latitude	Float	Google Maps	WGS84 decimal degrees, manually verified
Longitude	Float	Google Maps	WGS84 decimal degrees, manually verified
Opening hours	Text	Official listing	Structured time-window per day (e.g. 09:00–19:00)
Visit duration	Integer	Estimation	Expected time on-site in minutes
Google rating	Float	Google Maps	Aggregated user score on a 1–5 scale
TripAdvisor rating	Float	TripAdvisor	Aggregated user score on a 1–5 scale (missing for 18 sites)
Interest Score	Float	Computed	Google + TripAdvisor, or Google $\times$ 2 when TripAdvisor is missing, mapped to 1–10

3.2 Preprocessing

- **Coordinate verification:** All GPS coordinates were manually cross-checked using Google Maps satellite view to avoid positioning errors.
- **Interest score calculation:** An overall Interest Score was calculated on a 1–10 scale. When both ratings are available, the score is defined as Google + TripAdvisor; when only a Google rating exists (18 landmarks, 31%), the score is set to Google $\times$ 2, maintaining scale consistency.
- **Category assignment:** Each landmark was classified into one of five categories: historical, attraction, tradition & art, religious, and shopping. This enables interest-based filtering in the Day Planner application and improves the tourist experience.
- **Opening-hours standardisation:** Time windows used by the optimiser were converted into structured formats (e.g. 09:00–17:00). For venues with split opening hours (for example, 11:00–12:30 and 15:00–17:30), multiple time windows were defined for each day.

3.3 Descriptive Statistics and Characteristics of the Dataset

The dataset contains 58 unique landmarks divided into five categories. Numerical descriptive statistics are shown in Table 2, while the descriptive visualisations are shown in Figure 1 and Figure 2. Because two landmarks were marked as continuously closed, the visualisations are based only on the remaining 56 landmarks that are available to the planner.

The numerical summary is based on variables that directly influence recommendation and route optimisation: the calculated Interest Score, the two external rating sources, and the estimated visit duration. These statistics are used to measure the quality of the selected landmarks and the practicality of including them in daily itineraries.

Table 2: Descriptive statistics for numerical attributes.

Metric	Interest score	Google	TripAdvisor	Duration (h)
n	58	58	40	57
Mean	8.376	4.266	4.123	1.544
Median	8.600	4.400	4.200	1.500
Std dev	0.853	0.412	0.488	0.836
Min	5.600	3.000	2.000	0.500
Max	10.000	5.000	4.800	3.000

The Interest Scores range from 5.6 to 10.0 with an average of 8.38. This means that the data set is generally of high quality with good reviews. The Google and TripAdvisor ratings are highly correlated (averages of 4.27 and 4.12 respectively) which validates the summation-based scoring formula. The average visit duration of 1.54 h per landmark is directly fed to the optimiser when constructing feasible itineraries within a fixed time budget.

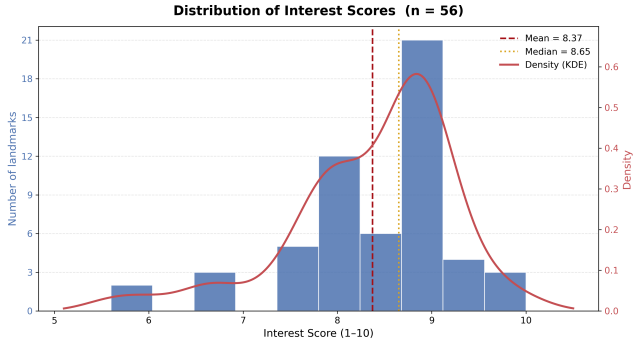

Figure 1: Distribution of Interest Scores with KDE overlay. Dashed = mean; dotted = median.

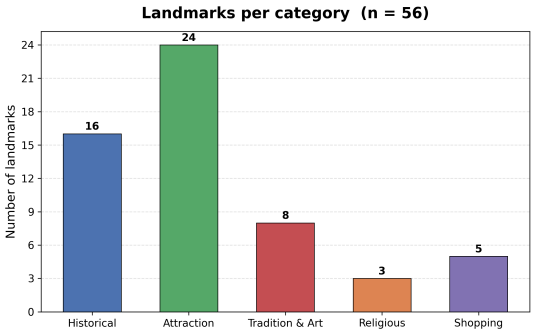

Figure 2: Landmark count per category for the 56 available landmarks.

4. Problem Solving Techniques

5. Application Design

The proposed system is architected as a modular, three-tier application designed to solve the **Orienteering Problem with Time Windows (OPTW)** within the urban context of Algiers. The design emphasizes the separation of concerns through a layered architecture, ensuring that the mathematical optimization logic remains decoupled from the service delivery and the user interface.

5.1. Core Logic Layer: AlgiersLIB

The foundation of the application is `AlgiersLIB`, a standalone Python library dedicated to modeling the problem domain and implementing optimization heuristics. To ensure extensibility, it utilizes the **Strategy Design Pattern**. An abstract base class, `Solver`, defines the interface for all optimization procedures, allowing the system to remain "open for extension but closed for modification."

5.2. Service Layer: Flask REST API

Built with the **Flask** framework, this layer manages data persistence, request validation, and the execution pipeline. Following the *KISS* principle, the system utilizes **CSV flat-files** for data storage, which is optimal for the static nature of the landmark dataset (≈ 50 points).

A critical performance optimization in this layer is the **Two-Phase Distance Calculation**. Because the meta-heuristics evaluate millions of potential routes, querying a real-world routing API during the search phase would be catastrophically slow and expensive. Instead, a **memoization-based caching system** computes a fast Haversine (straight-line) distance matrix for the solver loops. The Mapbox Directions API is strictly reserved for the final output phase to fetch accurate road geometries. Furthermore, a dedicated schema layer (`request_schemas.py`) ensures "Garbage In, Garbage Out" protection by validating user inputs before they reach the solvers.

5.3. Interface Layer: Frontend and UX

The frontend is developed using **Vanilla HTML5, CSS3, and JavaScript** to ensure a lightweight client-side footprint without the overhead of heavy frameworks. The UI is spatially divided to maximize usability: a left-hand configuration panel, a central interactive map canvas, a right-hand itinerary overlay, and a bottom KPI (Key Performance Indicator) dashboard.

A key feature of the interface is the preference weighting logic. The user visually ranks five specific landmark categories (**Attraction, Historical, Shopping, Religious, and Tradition/Art**). To dynamically adjust the importance of each category, the system converts the user-assigned rank (r) into a score multiplier (W_r) using an amplification coefficient ($C = 1.2$) across the total number of categories ($n = 5$):

$$W_r = \left(\frac{n-r}{n-1} - 0.5 \right) \times C + 1 \tag{1}$$

Using this formula, the ranks from 1 (best) to 5 (worst) map to a targeted multiplier spread of $\{\mathbf{1.6}, \mathbf{1.3}, \mathbf{1.0}, \mathbf{0.7}, \mathbf{0.4}\}$. These weights are passed directly to the `ScoringService` to adjust the objective function of the selected solver.

To gamify the user experience and abstract the mathematical complexity of the solvers for non-technical users, the system maps each of its optimization algorithms to a specific "Pokemon" persona reflecting its operational behavior. This roster spans simple greedy approaches (e.g., *Piplup*, *Bunnelby*), advanced meta-heuristics and evolutionary methods (e.g., *Mewtwo* for Tabu Search, *Eevee* for Genetic Algorithms), up to an exact IBM ILOG CPLEX solver (*Magnemite*) serving as the optimal baseline.

5.4. User Operational Flow

The interaction between the user and the three modules follows a deterministic workflow:

1. **Initialization:** In the left panel, the user defines the global constraints: the time window (start/end times), the starting hotel, and the specific day of the week (which enforces dynamic opening/closing hours for the OPTW constraints).
2. **Personalization:** The user ranks the five landmark categories via a drag-and-drop interface. The Frontend calculates the required weight multipliers (1.6 to 0.4) and selects a Solver.
3. **Request Orchestration:** The parameters are sent via an asynchronous AJAX request to the `/api/solve` endpoint.
4. **Backend Execution (Pipe-and-Filter Pattern):**
    - The `ProblemLoader` initializes constraints and validates the dataset.
    - The `ScoringService` applies the rank formula to compute a weighted score for every landmark.
    - The `SolverService` executes the selected strategy using the cached Haversine matrix for speed, operating under a strict **30-second maximum search time limit**.
    - The `RoutingService` fetches final road-network polylines from Mapbox for the optimized path.
5. **Visualization and Comparison:** After computation, the central Mapbox canvas renders the route. The bottom bar displays four primary KPIs: **Total Stops, Total Time, Distance (KM), and overall Interest Score**. The right-hand **Tour Guide** overlay provides a step-by-step itinerary, detailing each node's category, point value, and allocated visit duration. The solution is also added to a **Ranking Table**, allowing the user to seamlessly benchmark different algorithms.

Table 3: API Endpoint Specifications

Endpoint	Description
`/api/landmarks`	Retrieves the dataset of landmarks and metadata.
`/api/hotels`	Retrieves the list of available starting locations.
`/api/categories`	Returns landmark categories for user ranking.
`/api/solve`	Accepts constraints (budget, weights, algorithm) and returns an optimized tour.

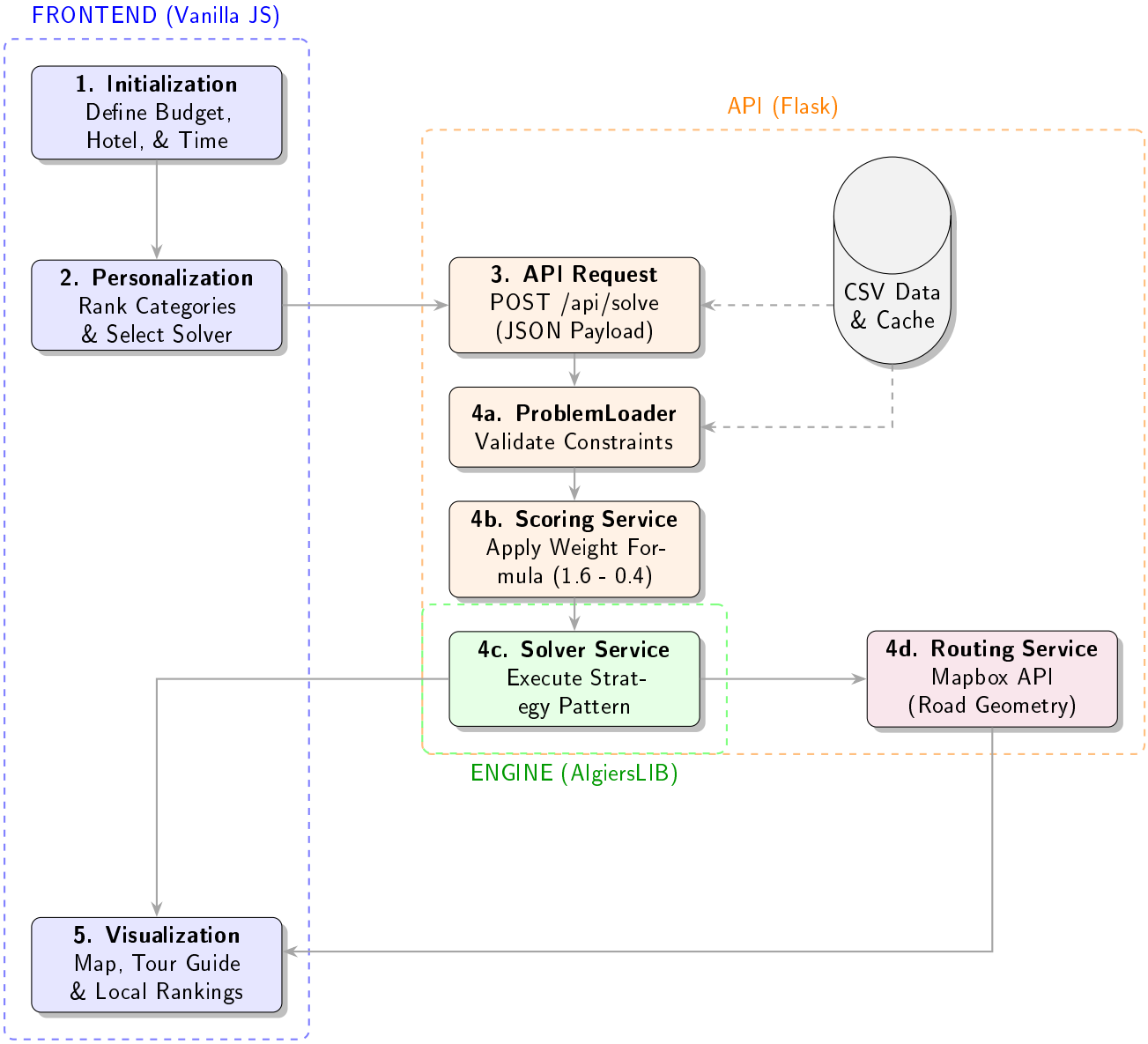

Figure 3: System Architecture of the Application.

6. Results and Analysis

6.1. Experimental Setup and Evaluation Methodology

All experiments were conducted on an AMD Ryzen 7 7435HS processor (3.10 GHz, 16 logical cores). Three datasets were used: (i) the team-collected **Algiers** dataset (59 real landmarks, multiple time windows); (ii) **Solomon-50 solomon50**, 50-node TOPTW benchmark instances (`c/r/rc_100_50`) covering clustered, random, and mixed layouts; and (iii) **Solomon-100 solomon100**, the full 100-node instances used for scalability stress-testing. CPLEX was excluded from Solomon-100 due to prohibitive runtime and capped at **30 s** on Algiers.

The evaluated solvers are: GREEDY (score- and ratio-priority variants), SA-BOLTZMANN, SA-CAUCHY, TABU, TABU-RANDOM, GRASP, GENETIC-TAILORED, GENETIC-SCORE, and CPLEX. Stochastic solvers ran **5 times per instance** (25 runs over 5 instances); deterministic solvers once. A multiprocessing framework with adaptive serialisation was used throughout.

Performance Metrics.

- **Score**: total collected interest along the tour.
- **Optimality gap** (%): $\text{Gap} = \max\left(0,\ \frac{\text{Optimal}-\text{Score}}{\text{Optimal}} \times 100\right)$, with CPLEX as ground truth for Solomon-50 **solomon50** and Righini & Salani **righini_salani** optima for Solomon-100 **solomon100**.
- **Execution time** (s): wall-clock time per run.
- **Optimal-finding rate**: fraction of stochastic runs matching the known optimum.

GENETIC-SCORE is excluded from optimality comparisons as its unconstrained fitness generates infeasible tours; it is discussed in Section 6.5.

6.2. Algiers Dataset

Figure 4 presents four performance views on the Algiers dataset. GRASP and TABU both attain the highest score of **98.7** (11 landmarks, 98.2% and 99.9% budget utilisation);however TABU attains the highest score under only 0.261 s the best speed–quality trade-off on this dataset. SA-CAUCHY (96.9) and SA-BOLTZMANN (95.3) follow closely. GENETIC-TAILORED achieves 90.0 across 11 landmarks in only 0.059 s—. The GREEDY variants diverge sharply: Ratio-Priority scores 70.5 (8 landmarks) while Score-Priority collapses to 19.2 (2 landmarks), confirming that score-greedy construction fails without feasibility enforcement.

Despite being an exact solver, CPLEX reaches only 88.2 under its 30 s cap— insufficient to prove optimality on a 59-node instance with heterogeneous time windows. TABU (98.7 at 0.216 s) and all top metaheuristics surpass it, illustrating the practical advantage of local-search methods under strict time budgets.

6.3. Solomon-50: Solution Quality

Figures 5 and 6 characterise solution quality on Solomon-50 **solomon50**. GENETIC-TAILORED achieves the lowest mean gap of **0.73%** (median 0.0%), matching the CPLEX optimum in **84%** of runs—the highest optimal-finding rate among all heuristics. GRASP attains 3.47% (60% optimal rate) with low variance. The Tabu variants (3.79% and 4.02%, rates of 48% and 40%) deliver near-optimal solutions in under 1 s—over 5× faster than GENETIC-TAILORED.

The SA variants exhibit high gaps (27.8% for SA-CAUCHY, 34.9% for SA-BOLTZMANN) and zero optimal-finding rate. Their cooling schedules—calibrated to iteration count rather than feasible neighbourhood size—cause premature convergence; the Cauchy schedule's heavier-tailed acceptance yields a ≈7 pp advantage over Boltzmann, but both remain uncompetitive. GREEDY anchors the bottom at a

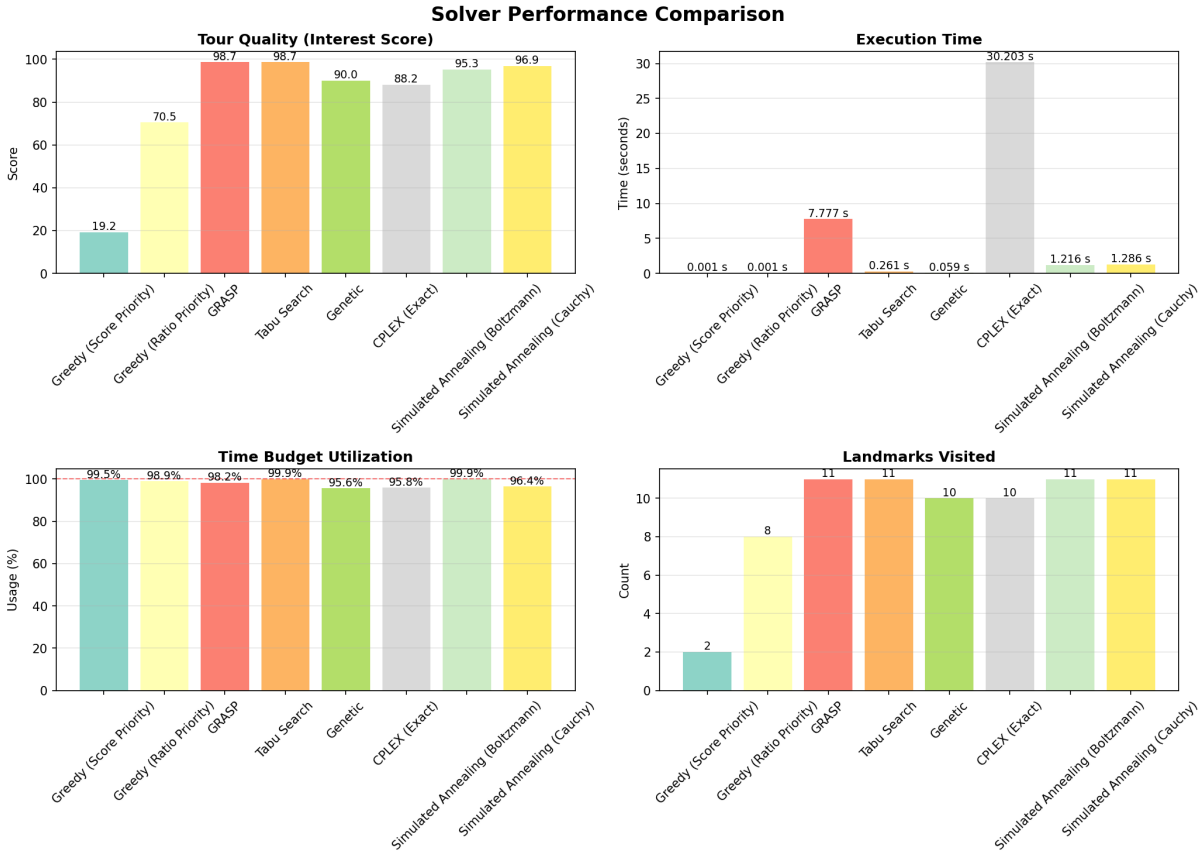

Figure 4: Solver performance on the Algiers real-world dataset (59 landmarks): tour quality (interest score), execution time, time-budget utilisation, and landmarks visited.

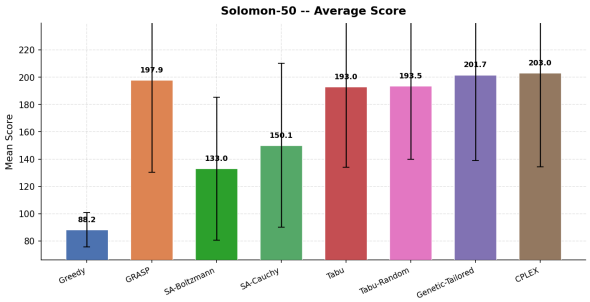

Figure 5: Score distributions on Solomon-50 **solomon50** (box plot over 25 stochastic / 5 deterministic runs).

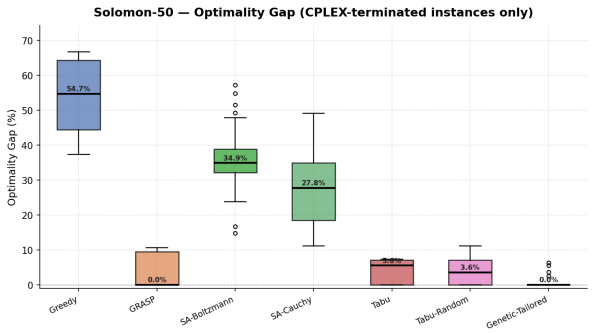

Figure 6: Optimality gap on Solomon-50 **solomon50** (CPLEX-terminated instances only).

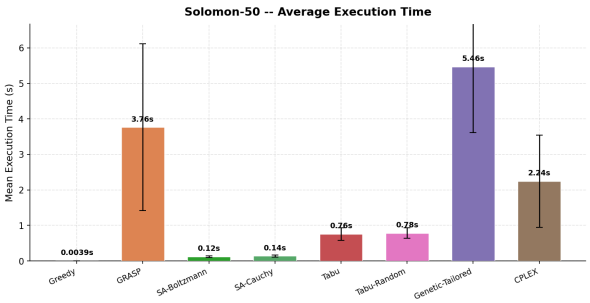

Figure 7: Mean execution time per solver on Solomon-50 **solomon50** (error bars: $\pm 1\sigma$).

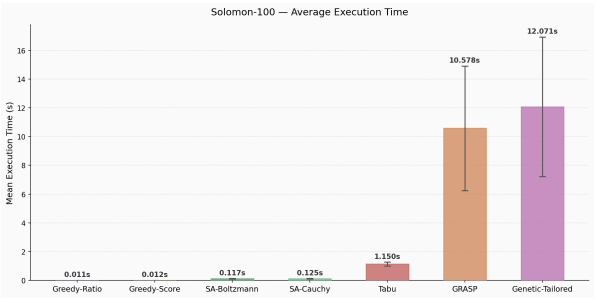

Figure 8: Mean execution time per solver on Solomon-100 **solomon100** (error bars: $\pm 1\sigma$).

54.7% median gap.

CPLEX closes optimally on all 5 instances (avg. 2.24 s), but runtime is highly instance-dependent: easy clustered cases finish in seconds while harder ones (e.g., `c102`) can take up to **60 minutes**, rendering it impractical at scale.

6.4. Solomon-100: Scalability Analysis

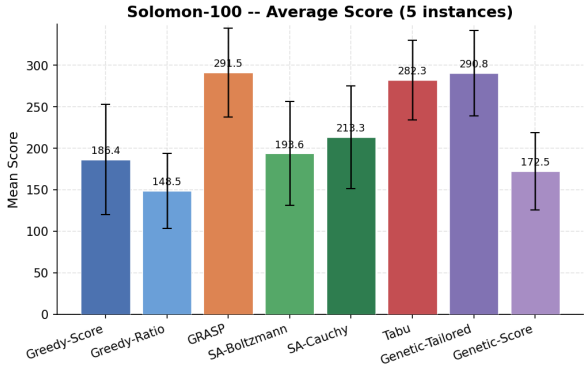

Figure 9: Score distributions on Solomon-100 **solomon100** (25 stochastic runs per solver).

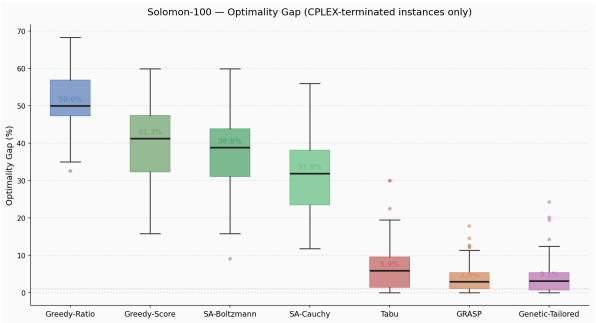

Figure 10: Optimality gap on Solomon-100 **solomon100** (Righini & Salani **righini_salani** ground truth).

Scaling to 100 nodes (Figures 9–8), the quality ordering is preserved but gaps widen. GRASP (3.95%) and GENETIC-TAILORED (3.98%) remain statistically tied, both sustaining sub-4% gaps against the Righini & Salani **righini_salani** optima. TABU degrades modestly to 6.59% but stays the fastest viable option at 1.15 s. SA and Greedy variants deteriorate significantly: SA-CAUCHY widens to 30.6%, SA-BOLTZMANN to 37.2%, and GREEDY-RATIO reaches 51.5%.

Execution times scale non-linearly: GRASP grows to 10.58 s ($\sigma = 4.33$ s) and GENETIC-TAILORED to 12.07 s ($\sigma = 4.85$ s), while SA and Tabu variants remain below 1.2 s. Table 4 provides a unified cross-scale summary.

Table 4: Consolidated benchmark results across Solomon-50 **solomon50** and Solomon-100 **solomon100**. Gap on Solomon-50 is relative to CPLEX; on Solomon-100 relative to Righini & Salani **righini_salani** optima. Opt. rate computed over 25 stochastic runs (Solomon-50 only). "–" denotes not applicable. Bold: best per column.

	Solomon-50			Solomon-100		
Algorithm	**Gap (%)**	**Time (s)**	**Opt. Rate**	**Gap (%)**	**Time (s)**	**Score**
Genetic-Tailored	**0.73**	5.46	**84%**	3.98	12.07	290.8
GRASP	3.47	3.76	60%	**3.95**	10.58	**291.5**
Tabu-Random	3.79	0.78	48%	–	–	–
Tabu	4.02	**0.76**	40%	6.59	**1.15**	282.3
SA-Cauchy	27.77	0.14	0%	30.57	0.12	213.3
SA-Boltzmann	34.86	0.12	0%	37.22	0.12	193.6
Greedy-Score	53.48	0.004	0%	39.92	0.01	186.4
Greedy-Ratio	–	–	–	51.48	0.01	148.5
CPLEX	0.00	2.24	100%	–	–	–

6.5. Key Insights and Trade-off Analysis

- **Quality–time Pareto front.** Solvers span a clear frontier: Greedy and SA variants are fast but sub-optimal (<0.15 s, gaps >27%), while GRASP and GENETIC-TAILORED are near-optimal but slower

(>3.5 s, gaps <4%). TABU-RANDOM uniquely dominates the mid-range—3.79% gap in under 1 s—making it the preferred choice for latency-sensitive deployment.

- **CPLEX: exactness versus practicality.** CPLEX is an indispensable calibration tool on Solomon-50 **solomon50**, but runtime is highly non-uniform: easy instances close in seconds while `c102` can take **60 minutes**. On Algiers, its 30 s cap yields 88.2—surpassed by GENETIC (90.0), SA-BOLTZMANN (95.3), SA-CAUCHY (96.9), and both GRASP/TABU (98.7).
- **SA cooling schedules.** Both SA variants suffer premature convergence because cooling is tied to iteration count rather than feasible neighbourhood size. Cauchy outperforms Boltzmann by ≈7 pp via heavier-tailed acceptance, but gaps grow with scale and neither variant ever finds the optimum.
- **GRASP vs. Genetic-Tailored.** Statistically indistinguishable gaps at both scales (<0.1 pp on Solomon-100 **solomon100**). GENETIC-TAILORED leads in optimal-finding rate (84% vs. 60% on Solomon-50 **solomon50**); GRASP is simpler and trivially parallelisable. The choice depends on engineering effort and the required frequency of finding the optimum.
- **Fitness design in constrained GAs.** GENETIC-SCORE's unconstrained fitness caused all Algiers runs and 76% of Solomon-50 **solomon50** runs to exceed the time budget, with scores up to 4× the CPLEX optimum (e.g., 970 vs. 180 on `rc101`). This is a structural failure: feasibility enforcement within the fitness function is a prerequisite for GAs on hard-constrained problems.
- **Instance-class sensitivity.** GENETIC-TAILORED leads on Solomon-50 **solomon50**; GRASP leads on Solomon-100 **solomon100**; both tie on Algiers. Clustered instances favour crossover-based search; random instances reward GRASP's greedy-randomised construction. This motivates instance-aware or hybrid solvers.

7. Discussion

The experimental results reveal a nuanced performance landscape in which solution quality, computational cost, and constraint satisfaction trade off in ways that both confirm and extend the findings reported in the orienteering literature.

7.1. Interpretation of Results

The most striking result is the parity between GRASP and Tabu Search at the highest observed tour quality of 98.7, both visiting 11 landmarks within the eight-hour budget, yet with a wall-clock time ratio of approximately 27:1 in favour of Tabu Search (0.37 seconds versus 10.18 seconds). This finding is consistent with the comparative analysis of **Liang2002<empty citation>**, who observed that Tabu Search and population-based heuristics achieve comparable solution quality on orienteering benchmarks, and with **Tang2005<empty citation>**, who demonstrated that adaptive memory mechanisms in Tabu Search enable rapid convergence through structured neighbourhood pruning. The successive-filtering candidate strategy implemented in our Tabu Search — ranking landmarks by induced travel cost relative to interest score — appears to be the principal factor responsible for its computational efficiency, effectively reducing the neighbourhood evaluation burden without sacrificing solution quality.

Simulated Annealing with the Cauchy acceptance criterion (96.9, 11 landmarks) consistently outperforms the Boltzmann variant (95.3, 11 landmarks), a result attributable to the heavier tails of the Cauchy distribution, which sustain a higher probability of accepting deteriorating moves at intermediate temperatures and thereby reduce premature convergence. This echoes the theoretical expectations discussed by **Gunawan2016<empty citation>**, who note that acceptance-function design is a primary differentiator of Simulated Annealing variants on constrained routing problems.

The Genetic Algorithm (68.3–90.0 across configurations) exhibits greater variance than single-solution methods, which is consistent with the sensitivity of population-based approaches to crossover operator design on time-windowed instances. The order crossover and tailored crossover implementations produced different fitness landscapes, and the relatively modest performance of the penalty fitness function relative to the feasibility-oriented variant suggests that soft-constraint handling significantly affects search trajectory for this problem class.

The most academically interesting result is the underperformance of the CPLEX exact solver (61.8–88.2, 7–10 landmarks) relative to GRASP and Tabu Search. Three explanations are plausible. First, CPLEX required 30.2 seconds to terminate, suggesting that the branch-and-bound search tree was not fully explored within the available computation budget, and the best integer solution found at termination is consequently a feasible but sub-optimal incumbent. Second, the Mixed Integer Linear Programming formulation introduces symmetry constraints and big-M linearisations for time-window propagation that tighten the feasible region relative to the heuristic representations. Third, the haversine-based travel matrix, while consistent across all solvers, produces a non-integer cost structure that may slow LP relaxation bounds. **Righini2009<empty citation>** observed similar behaviour and addressed it through decremental state-space relaxation; incorporating such techniques would likely close the gap between CPLEX and the leading metaheuristics.

7.2. Limitations

Several limitations constrain the generalisability of these results. First, travel times are computed from straight-line haversine distances at a fixed average urban speed of 25 km/h. Real road distances in Algiers, which the backend routing service retrieves from MAPBOX API, are systematically longer and temporally variable due to traffic. The algorithms were therefore optimising a surrogate objective that diverges from the true tourist experience. Second, landmark interest scores were assigned through a weighted combination of public ratings and expert judgement, a process that introduces subjectivity and limits reproducibility. Third, the dataset is confined to 56 landmarks in the Algiers wilaya, a scale at

which all tested algorithms remain tractable; results may not extrapolate to larger metropolitan datasets. Finally, the time-window representation assumes deterministic opening hours, whereas real tourist sites may close unexpectedly or have dynamic crowd-dependent effective visit durations.

7.3. Future Work

Several directions offer meaningful improvements. Within the classical metaheuristic paradigm, Adaptive Large Neighbourhood Search (ALNS) represents the current state of the art for time-constrained orienteering (**Gunawan2016**), with its self-tuning destroy-and-repair operators addressing the principal weakness of both GRASP (construction diversity) and Tabu Search (neighbourhood breadth). Iterated Local Search with the MaxShift data structure of **Vansteenwegen2009<empty citation>** would provide an efficient feasibility oracle that reduces the per-iteration cost of neighbourhood evaluation from $O(n)$ to $O(1)$.

At the frontier of the field, deep reinforcement learning (DRL) approaches have demonstrated near-optimal performance on routing problems of this scale. **Kool2019<empty citation>** showed that an Attention Model trained with the REINFORCE policy gradient algorithm learns generalisable construction heuristics for TSP and related problems; **Gama2020<empty citation>** extended this framework directly to the OPTW. Hybrid DRL-ALNS architectures, in which a learned policy selects neighbourhood operators dynamically (**Reijnen2024**), represent the most promising direction for achieving both solution quality and sub-second inference times. A further extension of direct practical relevance is the Stochastic Orienteering Problem, in which travel times are drawn from a distribution reflecting real traffic conditions; this formulation is underexplored in the literature (**Vansteenwegen2011**) and maps naturally onto the Algiers urban context. Finally, extending the single-day formulation to the Team Orienteering Problem with Time Windows (**Gunawan2016**) would enable multi-day, multi-guide itinerary planning, substantially increasing the practical utility of the system.

8. Conclusion

This paper presented *Algiers in 24 Hours*, a tourist route optimisation system grounded in the Time-Constrained Orienteering Problem with Time Windows. Six algorithmic approaches spanning the full spectrum from deterministic construction to exact branch-and-bound were implemented, integrated into a common model layer, and evaluated on a curated dataset of 56 Algiers landmarks with verified GPS coordinates, opening hours, and interest scores.

The principal finding is that Tabu Search with Strategic Oscillation achieves solution quality equivalent to GRASP (98.7 interest score, 11 landmarks) at a fraction of the computational cost (0.37 versus 10.18 seconds), establishing it as the recommended solver for real-time deployment contexts. Simulated Annealing with the Cauchy acceptance criterion provides a competitive alternative (96.9) with moderate runtime. The underperformance of CPLEX relative to leading metaheuristics underscores the known intractability of the OPTW for branch-and-bound approaches under realistic time budgets, and motivates the design of hybrid exact-heuristic methods as a direction for future investigation.

Beyond algorithmic contribution, this work produces a reusable, open-source problem model class for the Algerian tourist orienteering context — a domain absent from existing benchmark libraries — and a full-stack web application through which algorithm behaviour can be explored interactively by non-expert users. The gap between the best heuristic score and the CPLEX incumbent across all experiments defines a concrete quality target for future work. Closing that gap through Adaptive Large Neighbourhood Search or learned construction heuristics, while extending the formulation to multi-day and stochastic settings, constitutes the principal agenda for subsequent research.

A. Screenshots of the System

Appendix A: Application Screenshots

The following screenshots illustrate the web application developed as part of this project, covering the main interface, configuration panels, solver execution, and result visualisation.

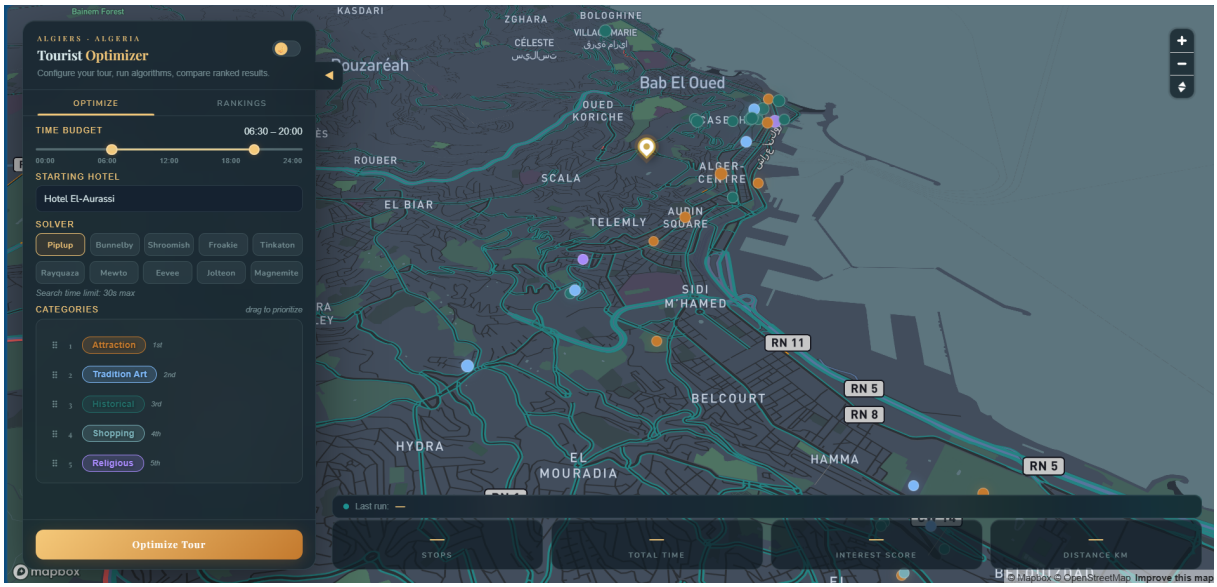

Figure 11: Main interface of the tourism itinerary planning application in dark mode.

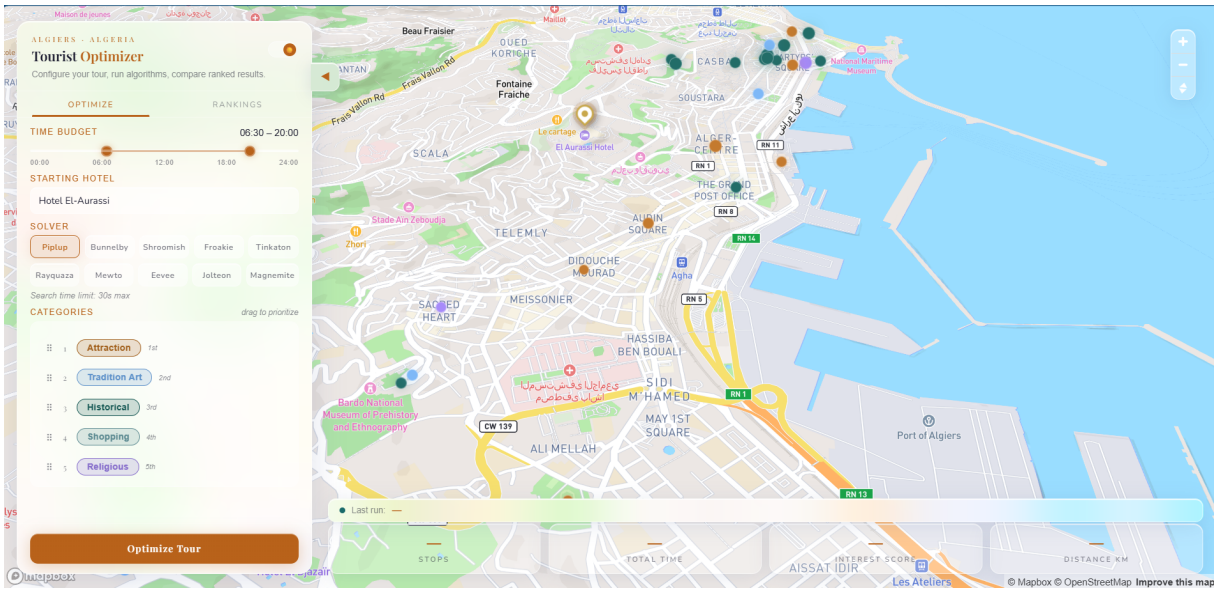

Figure 12: Light mode variant of the application interface, offering an alternative visual theme.

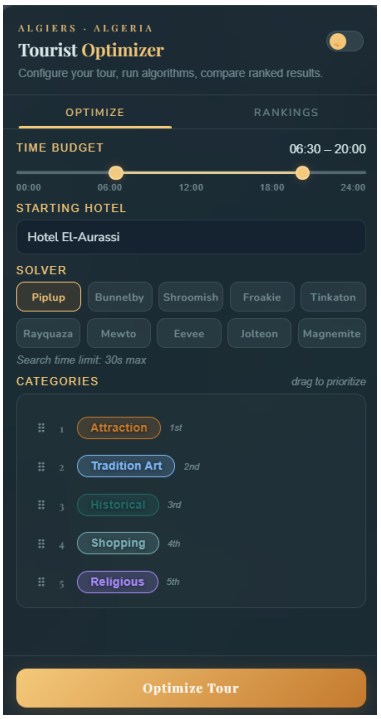

Figure 13: Control panel for configuring the time budget, travel speed, and departure point.

Figure 14: Drop-down menu for selecting the starting hotel or departure location.

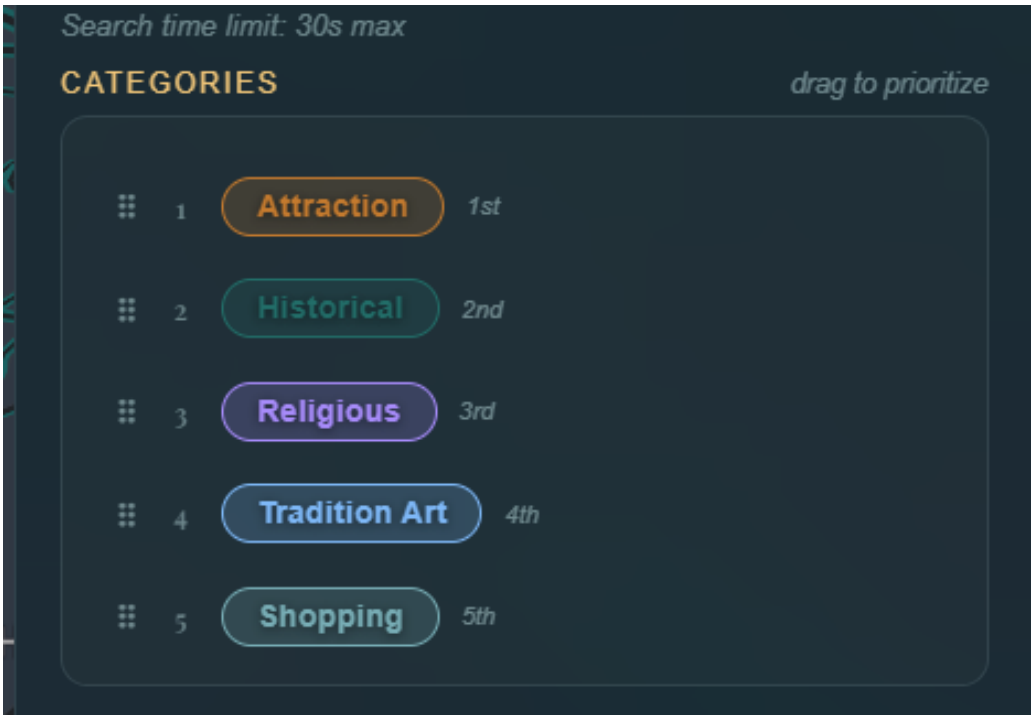

Figure 15: Drag-and-drop interface for reordering priority among landmark categories.

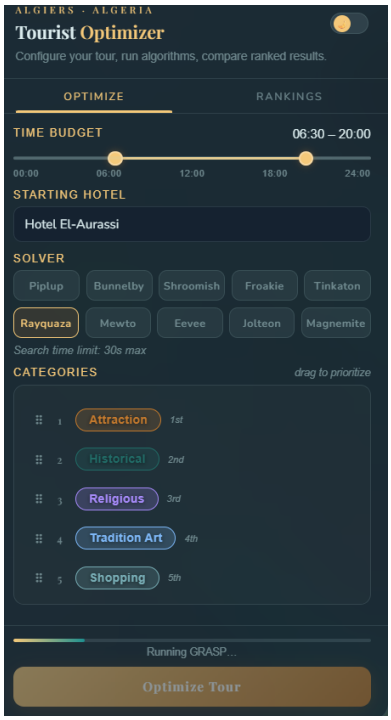

Figure 16: Solver selection panel listing the available optimisation algorithms to execute.

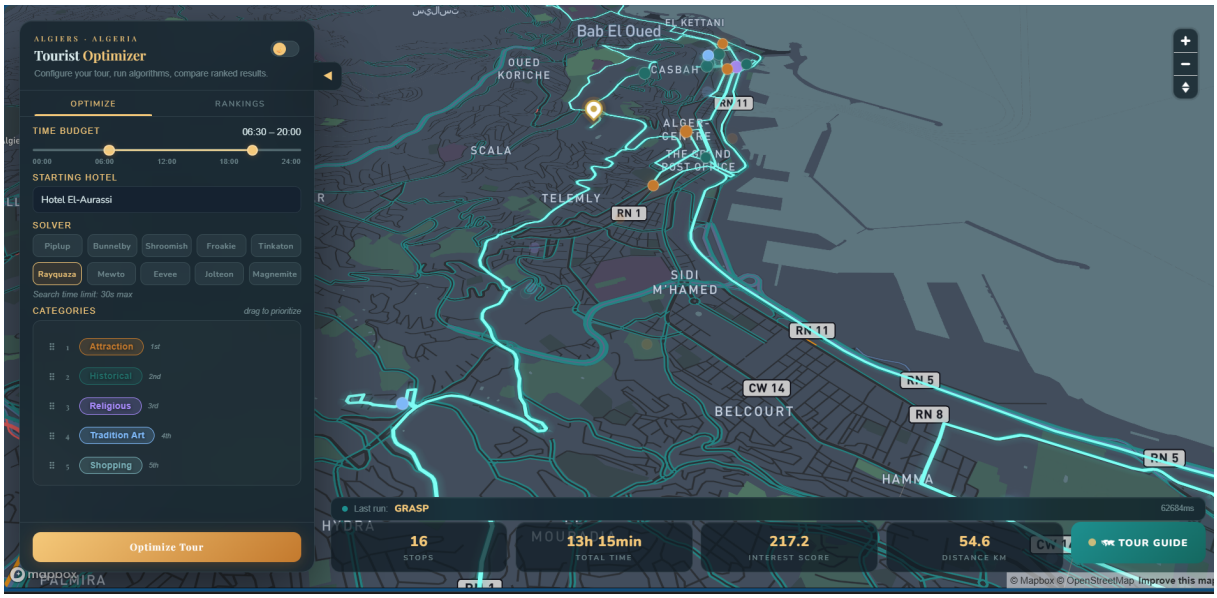

Figure 17: Optimised tour route computed by GRASP, rendered on an interactive map.

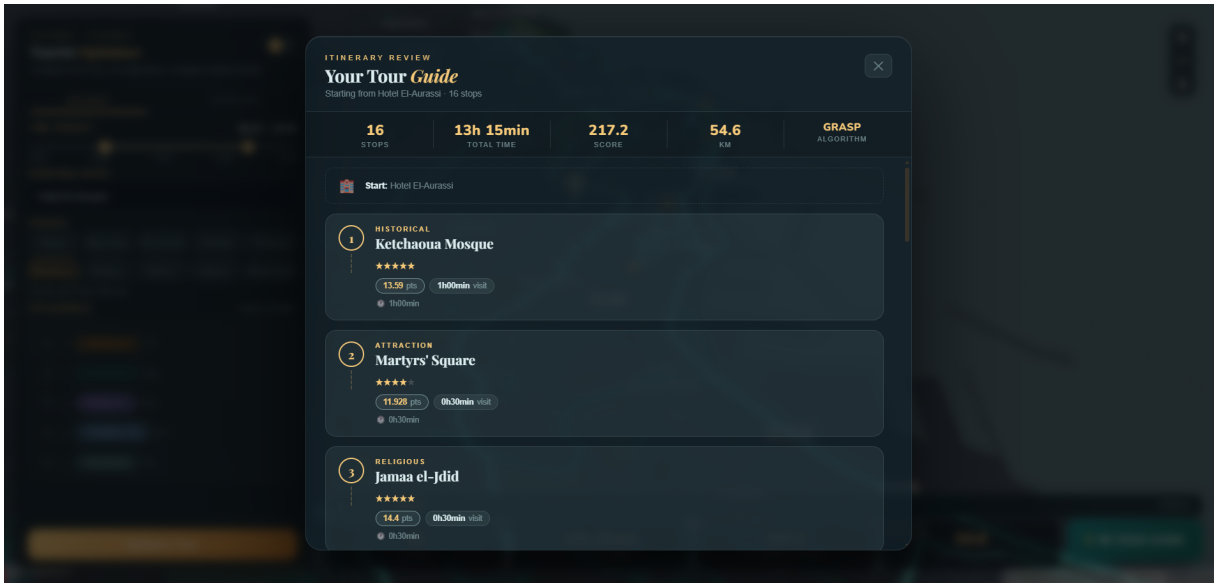

Figure 18: Detailed summary of the best tour found by GRASP: visited landmarks, scores, and timing.

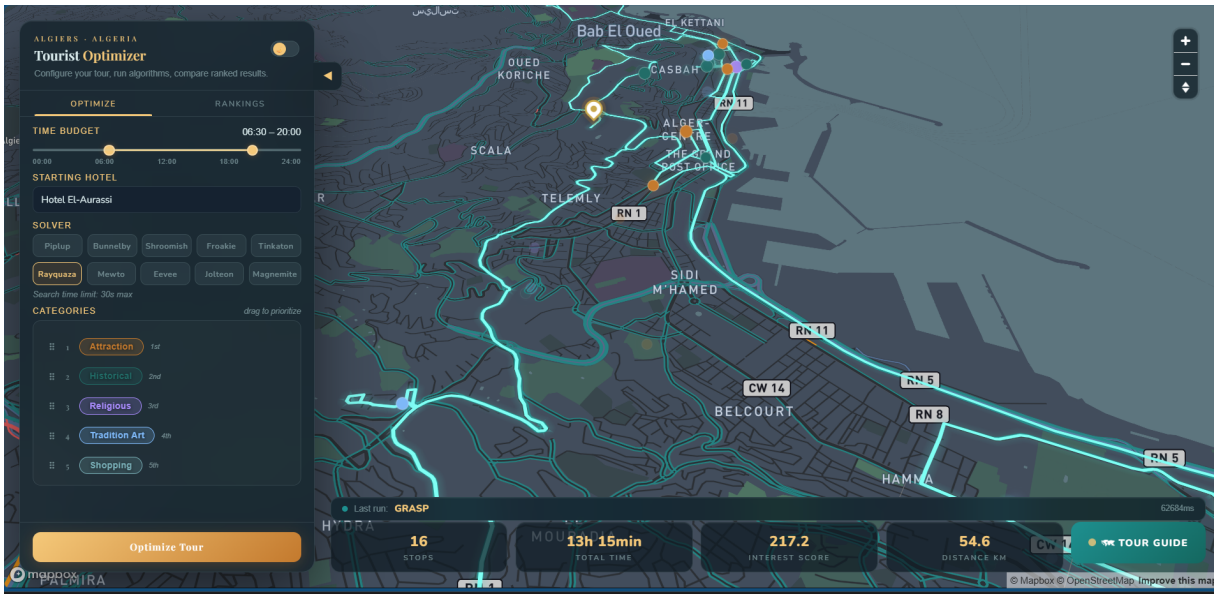

Figure 19: Full map view of the GRASP-computed itinerary with route path overlay.

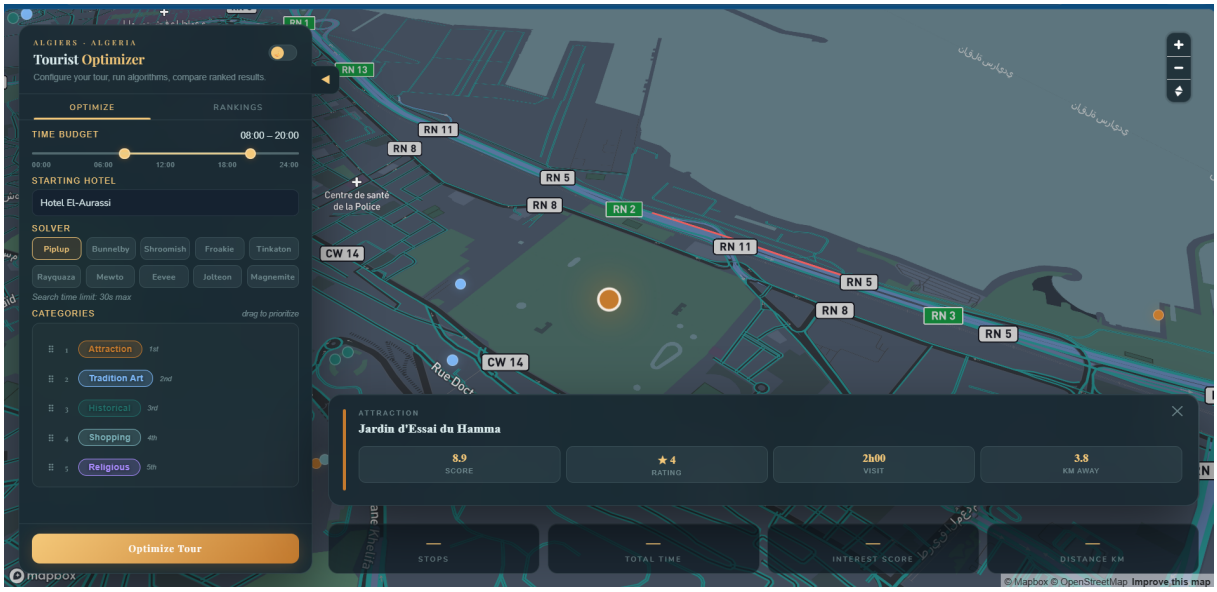

Figure 20: Landmark information tooltip displayed on map hover: name, category, and time window.

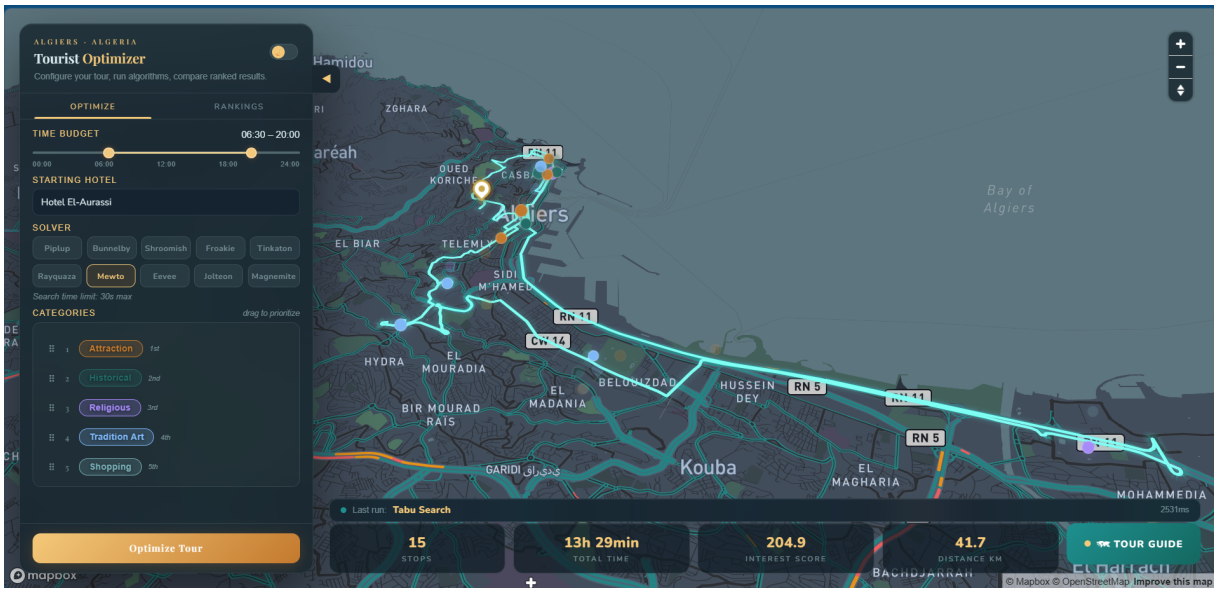

Figure 21: Optimised itinerary produced by Tabu Search displayed on the map.

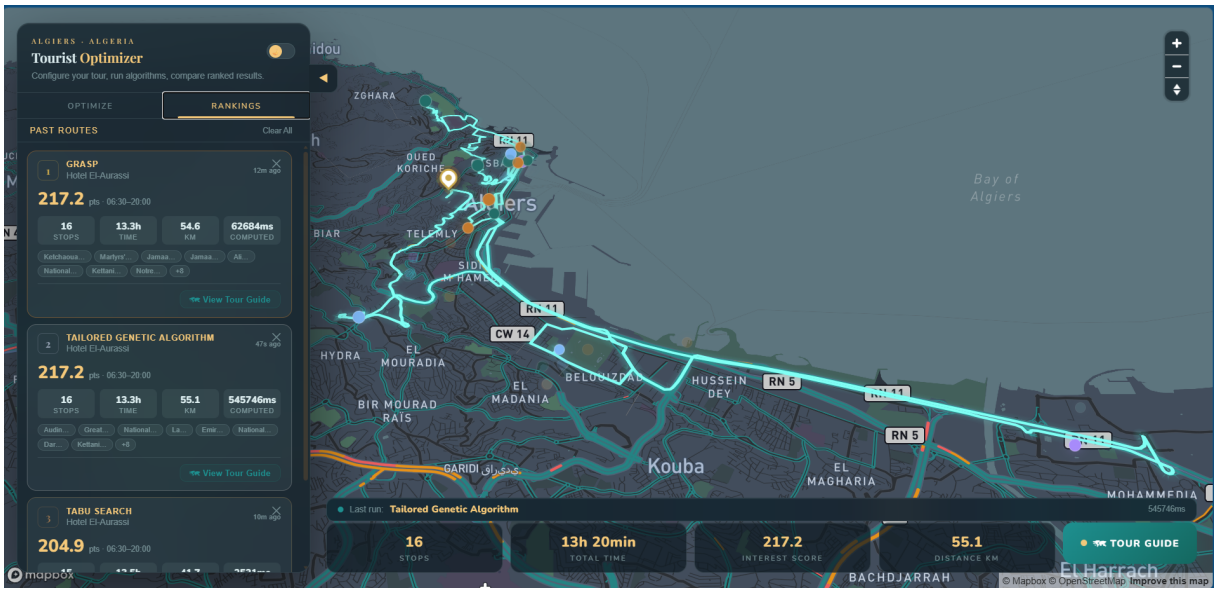

Figure 22: Algorithm performance ranking after multiple solver runs on the selected instance.

B. Who Did What

Table 5: Team member contributions.

Member	Primary Contributions
Raouf Ould Ali	Integrate Frontend, Implement Simulated Annealing, Implement CPLEX Solution, Setup GitHub Repository, Review Performance Analysis, Review Grasp Search, Write Application Design (Report), Review Dataset Building (Report), Review Conclusion (Report), Write References (Report).
Mohamed Zakaria Razam	Implement Frontend, Implement Greedy Search Algorithm, Write Code Documentation, Review Simulated Annealing, Review CPLEX Solution, Write Dataset Building (Report), Review Application Design (Report), Review Discussion (Report), Write Appendix B (Report).
Ahmed Adnane Meddah	Review Frontend, Implement Performance Analysis & Visualization, Implement Unit Testing, Review Greedy Search, Write Introduction (Report), Review Appendix A (Report), Review Appendix B (Report), Review Jupyter Notebook.
Mohamed Charaf Eddine Deghboudj	Backend Development, Implement Problem Model Class, Implement Grasp Search, Review Genetic Algorithm, Review Tabu Search, Review State of the Art (Report), Review Results and Analysis (Report), Write Discussion (Report), Write Conclusion (Report).
Yacine Mouloud	Backend Development, Implement Problem Model Class, Implement Tabu Search, Implement Unit Testing, Review Abstract (Report), Review Problem Solving Techniques (Report), Write Results and Analysis (Report), Write Appendix A (Report).
Aboubakr Bendahmane	Implement Genetic Algorithm, Setup LaTeX Template, Review Problem Model, Review Unit Testing, Write Abstract (Report), Write State of the Art (Report), Write Problem Solving Techniques (Report), Review Introduction (Report), Review References (Report).